



REDES NEURONALES Y APRENDIZAJE PROFUNDO

Nombres: MACÍAS MONDRAGÓN MARCOS RODLFO, REYES CRUZ ROBERTO MISAEL

Especialidad: Ing. en Inteligencia Artificial

Fecha de la práctica: 12 de abril del 2026

Nombre de la práctica: Explorando la Red de Hopfield

Objetivo:

Familiarizarse con el entrenamiento y el comportamiento de una red de Hopfield utilizando patrones binarios. Los estudiantes implementarán la red, la entrenarán con diferentes patrones, y explorarán cómo la red converge hacia uno de los patrones entrenados.

Instrucciones:

1. Implementación de la Red de Hopfield:

- Usa el código visto en clase para la red de Hopfield.
- Revisen el código para entender cómo se inicializan los pesos y cómo se realiza la actualización asincrónica de los estados de las neuronas.

2. Entrenamiento con Nuevos Patrones:

- Entrena la red con al menos 8 patrones binarios distintos. Se pueden usar mas letras o números.
- Los patrones deben ser suficientemente distintos entre sí para evitar confusión.

3. Agregar Ruido a los Patrones:

- Generar versiones corruptas de los patrones entrenados (prueben para 4 diferentes patrones, cada uno con 15%, 25% y 35%).
- Ejecuta la red de Hopfield con los patrones corruptos y observa si la red es capaz de restaurarlos al patrón original entrenado.

4. Análisis de Convergencia:

- Modifiquen el código para mostrar el estado de la red cada iteración.
- Realicen un análisis sobre cómo la red converge hacia uno de los patrones entrenados. responde las siguientes preguntas:
 - ¿Cómo afecta el ruido en los patrones a la convergencia de la red?
 - ¿Qué ocurre si el ruido es demasiado grande (ej. cambia más del 50% de los bits)?
 - ¿La red siempre converge a un patrón entrenado? ¿Qué pasa si no lo hace?
 - Agreguen más patrones a la red y exploren cómo la red responde a un mayor número de patrones entrenados.

- Discute sobre la **capacidad de la red** (cuántos patrones puede memorizar sin confundirlos).

Muestra las evidencias de funcionamiento.

Elabora las conclusiones, haz énfasis en posibles aplicaciones interesantes de este tipo de redes.

Incluye las referencias

Parte 2: Entrenamiento con Nuevos Patrones

```
# Patrones 8x8 para A,C,S,T,V,M,/,+
patrones = {
    'A': np.array([
        [-1,-1, 1, 1, 1, 1,-1,-1],
        [-1, 1,-1,-1,-1,-1, 1,-1],
        [ 1,-1,-1,-1,-1,-1, 1],
        [ 1, 1, 1, 1, 1, 1, 1, 1],
        [ 1,-1,-1,-1,-1,-1, 1],
        [ 1,-1,-1,-1,-1,-1, 1],
        [ 1,-1,-1,-1,-1,-1, 1],
        [ 1,-1,-1,-1,-1,-1, 1],
        [-1,-1,-1,-1,-1,-1,-1,-1]
    ]).flatten(),

    'C': np.array([
        [-1, 1, 1, 1, 1, 1,-1,-1],
        [ 1, 1,-1,-1,-1,-1, 1],
        [ 1,-1,-1,-1,-1,-1,-1],
        [ 1,-1,-1,-1,-1,-1,-1],
        [ 1,-1,-1,-1,-1,-1,-1],
        [ 1,-1,-1,-1,-1,-1,-1],
        [ 1, 1,-1,-1,-1,-1, 1],
        [-1, 1, 1, 1, 1, 1,-1,-1]
    ]).flatten(),

    'S': np.array([
        [ 1, 1, 1, 1, 1, 1, 1, 1],
        [ 1,-1,-1,-1,-1,-1,-1,-1],
        [ 1, 1, 1, 1, 1, 1,-1,-1],
        [-1,-1,-1,-1,-1,-1,-1, 1],
        [-1,-1,-1,-1,-1,-1,-1, 1],
        [ 1, 1, 1, 1, 1, 1,-1,-1],
        [-1,-1,-1,-1,-1,-1,-1,-1],
        [ 1, 1, 1, 1, 1, 1, 1, 1]
    ]).flatten(),

    'T': np.array([
        [ 1, 1, 1, 1, 1, 1, 1, 1],
        [-1,-1,-1, 1, 1,-1,-1,-1],
        [-1,-1,-1, 1, 1,-1,-1,-1],
        [-1,-1,-1, 1, 1,-1,-1,-1],
        [-1,-1,-1, 1, 1,-1,-1,-1],
        [-1,-1,-1, 1, 1,-1,-1,-1],
        [-1,-1,-1, 1, 1,-1,-1,-1],
        [-1,-1,-1, 1, 1,-1,-1,-1]
    ]).flatten(),

    'V': np.array([
        [ 1,-1,-1,-1,-1,-1,-1, 1],
        [ 1,-1,-1,-1,-1,-1,-1, 1],
        [ 1,-1,-1,-1,-1,-1,-1, 1],
        [ 1,-1,-1,-1,-1,-1,-1, 1],
        [-1, 1,-1,-1,-1,-1, 1,-1],
        [-1,-1, 1,-1,-1, 1,-1,-1],
        [-1,-1,-1, 1, 1,-1,-1,-1],
        [-1,-1,-1,-1,-1,-1,-1,-1]
    ]).flatten(),

    'M': np.array([
        [ 1,-1,-1,-1,-1,-1,-1, 1],
        [ 1, 1,-1,-1,-1,-1, 1, 1],
        [ 1,-1, 1,-1,-1, 1,-1, 1],
        [ 1,-1,-1, 1, 1,-1,-1, 1],
        [ 1,-1,-1,-1,-1,-1,-1, 1],
        [ 1,-1,-1,-1,-1,-1,-1, 1],
        [ 1,-1,-1,-1,-1,-1,-1, 1],
        [ 1,-1,-1,-1,-1,-1,-1, 1]
    ]).flatten(),

    '/': np.array([
        [-1,-1,-1,-1,-1,-1, 1,-1],
        [-1,-1,-1,-1,-1, 1,-1,-1],
        [-1,-1,-1,-1, 1,-1,-1,-1],
        [-1,-1,-1, 1,-1,-1,-1,-1],
        [-1,-1, 1,-1,-1,-1,-1,-1],
        [-1, 1,-1,-1,-1,-1,-1,-1],
        [ 1,-1,-1,-1,-1,-1,-1,-1],
        [-1,-1,-1,-1,-1,-1,-1,-1]
    ]).flatten(),

    '+': np.array([
        [-1,-1,-1, 1, 1,-1,-1,-1],
        [-1,-1,-1, 1, 1,-1,-1,-1],
        [-1,-1,-1, 1, 1,-1,-1,-1],
        [ 1, 1, 1, 1, 1, 1, 1, 1],
        [ 1, 1, 1, 1, 1, 1, 1, 1],
        [-1,-1,-1, 1, 1,-1,-1,-1],
        [-1,-1,-1, 1, 1,-1,-1,-1],
        [-1,-1,-1, 1, 1,-1,-1,-1]
    ]).flatten(),
}
```

Figura 1. Nuevos Patrones

Parte 3: Agregar Ruido a los Patrones:

Para la letra A:

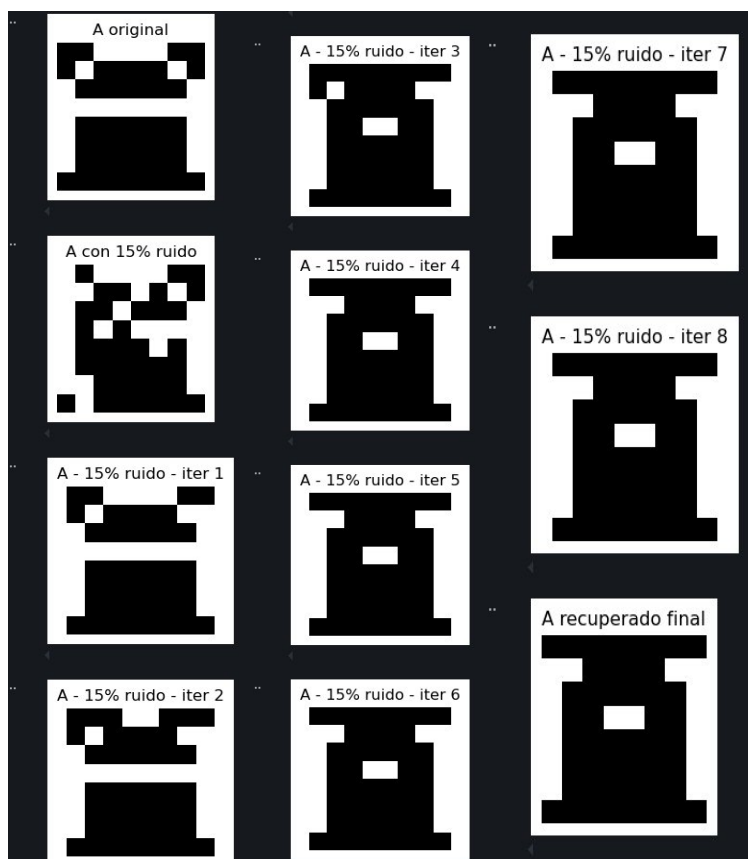


Figura 2. Letra A con 15% de ruido.

Para la letra C:

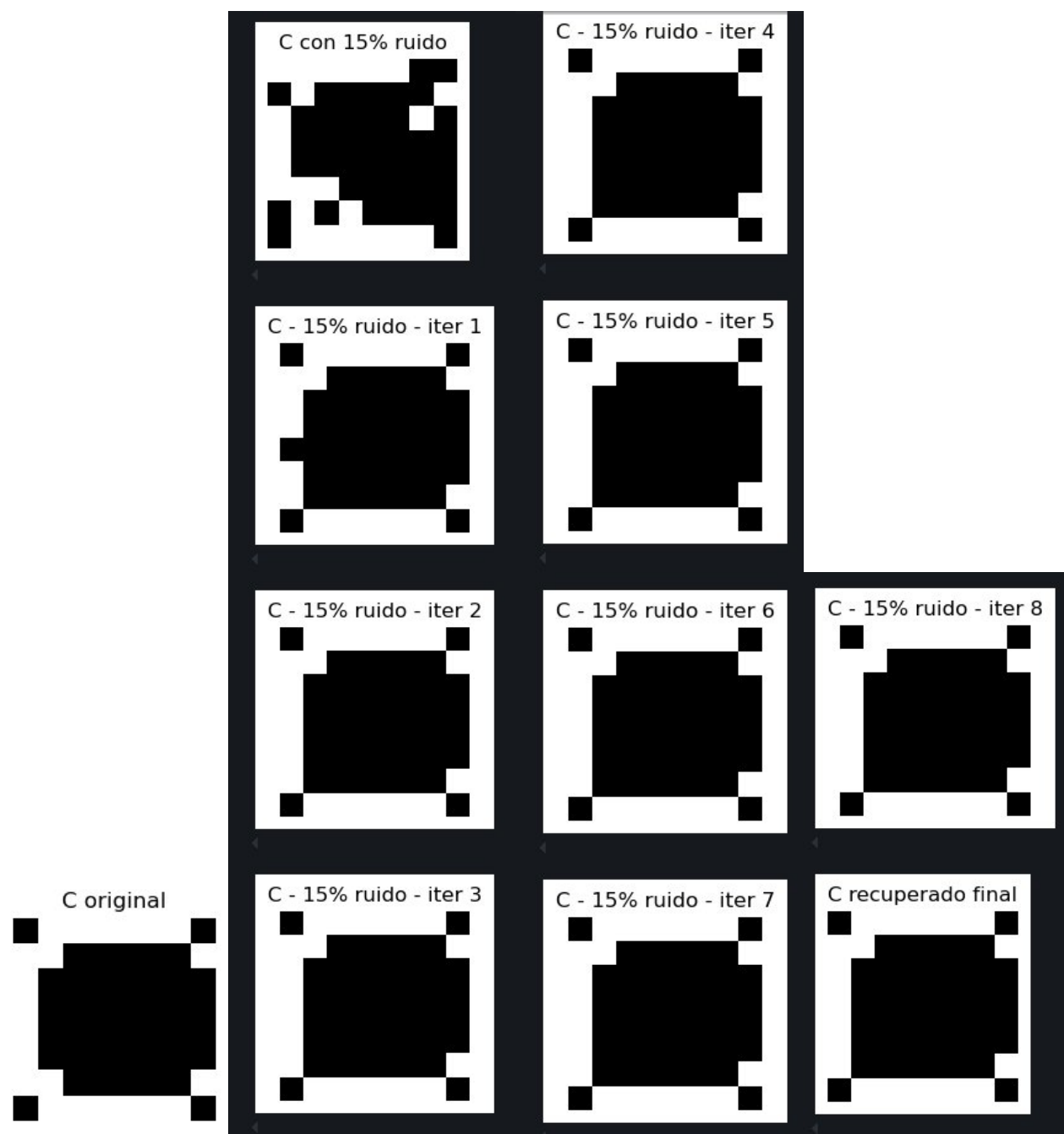


Figura 3: Letra C con 15% de Ruido

Para el símbolo +:

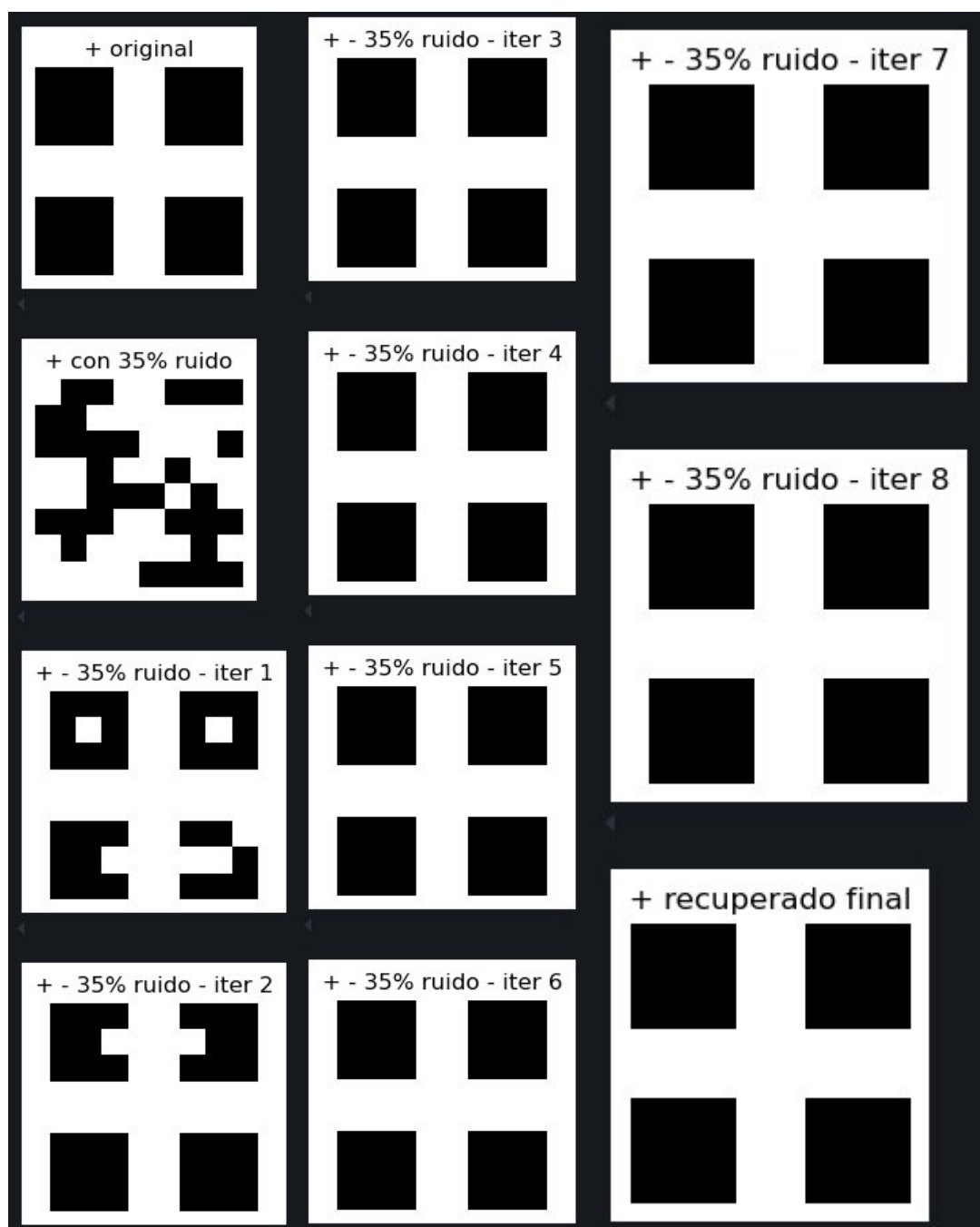


Figura 4: Símbolo + con 35% de Ruido

Para el símbolo / :

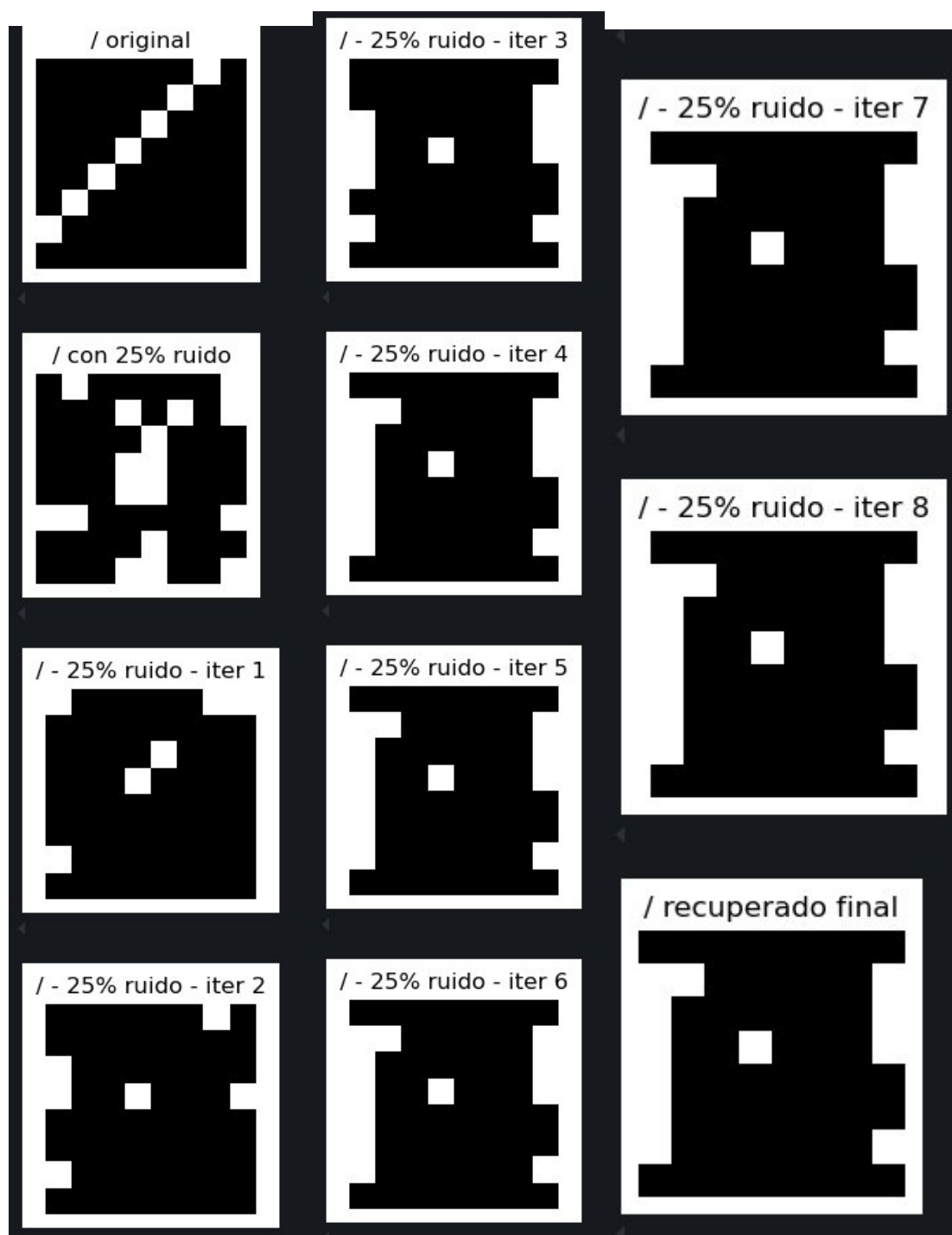


Figura 5: Símbolo / con 25% de Ruido

Parte 4: Análisis de Convergencia

- Realicen un análisis sobre cómo la red converge hacia uno de los patrones entrenados. responde las siguientes preguntas:
 - ¿Cómo afecta el ruido en los patrones a la convergencia de la red?

El ruido introduce errores en los bits del patrón de entrada, alejándolo del patrón original almacenado. A medida que aumenta el porcentaje de ruido, la red tarda más iteraciones en converger, la recuperación puede volverse menos precisa y existe mayor probabilidad de converger a un patrón incorrecto.

En niveles moderados la red suele recuperar correctamente el patrón original. En niveles más altos, la recuperación puede volverse inestable o incorrecta.

- ¿Qué ocurre si el ruido es demasiado grande (ej. cambia más del 50% de los bits)?

El patrón de entrada pierde su similitud con el original lo que ocasiona que la red puede:

- Converger a otro patrón almacenado
- Converger a un estado espurio (no entrenado)
- No estabilizarse correctamente

Esto ocurre porque la red funciona como un sistema de memoria asociativa basado en similitud.

- ¿La red siempre converge a un patrón entrenado? ¿Qué pasa si no lo hace?

No, la red no siempre converge a un patrón entrenado. Ya que tiende a converger a un patrón espurio, se quede en un estado incorrecto pero estable o que directamente converja al patrón equivocado. Esto dependerá de la cantidad de patrones almacenados, el nivel de ruido y la similitud entre patrones.

Conclusiones

Se observó que la red no siempre recupera exactamente el patrón original, incluso con niveles moderados de ruido. Esto se debe a la presencia de estados espurios y a la interferencia entre patrones no ortogonales, lo que provoca que la red converja a configuraciones similares pero incorrectas. En cambio, en el caso de “C” y “+” si pudo recuperar casi la forma original, ya que estos son ortogonales, pero no a todos los niveles de ruido (véase el Notebook).

Aplicaciones

- Reconocimiento de patrones (letras, números)
- Restauración de imágenes dañadas
- Sistemas de memoria asociativa
- Corrección de errores en señales
- Modelos simplificados de memoria biológica

Referencias

[1] J. J. Hopfield, “Neural networks and physical systems with emergent collective computational abilities,” Proc. Natl. Acad. Sci., vol. 79, no. 8, pp. 2554–2558, 1982.

[2] S. Haykin, Neural Networks and Learning Machines, 3rd ed. Pearson, 2009.

[3] C. M. Bishop, Pattern Recognition and Machine Learning. Springer, 2006.